

multitrack_work_binding

multitrack_work_binding.sh

Revision: 2 **Last modified:** 2026-07-10T22:10:00Z **Authority:** constitution §11.4.191 (work-to-track/branch binding enforcement) **Maintainer:** constitution submodule (inherited by reference per §11.4.177 / §11.4.28(B)) **Scope:** §11.4.18 companion doc for `constitution/scripts/multitrack/multitrack_work_binding.sh`

Overview

`multitrack_work_binding.sh` is the shared, project-agnostic **resolver** that answers the single §11.4.191 question:

Does this changed-file set (or this ticket) belong to a logic-group whose canonical branch/track is **not** the current checkout?

It is the one code path called by both §11.4.191 entry points:

- the PreToolUse guard hook `guard-work-track-binding.sh` (PREVENTIVE), and
- the commit wrapper / pre-build gate `CM-WORK-TRACK-BINDING-ENFORCED` (DETECTIVE).

Each input is mapped to its owning logic-group via the `group_paths` file-scope manifest (**longest-glob**, most-specific wins) or via `items.logic_group` for a ticket, then checked against `logic_groups.destination` (authoritative branch) + `logic_groups.canonical_track`.

Prerequisites

- `sqlite3` on `PATH` (absent $\Rightarrow$ fail-closed BLOCK, §11.4.6). **No `jq`**.
- `git` — only when `--staged` (or `cwd-track` derivation) is used.
- The §11.4.93/§11.4.95 workable-items DB at schema **v4**
(`logic_groups.canonical_track`
 - `group_paths`). Resolution order: `--db P` $\rightarrow$ `$WI_DB` $\rightarrow$ **superproject-walk outer roots** (see "Submodule-context DB resolution" below — a call from INSIDE a submodule finds the PARENT project's registry) $\rightarrow$ `<git-top>/docs/workable_items.db` $\rightarrow$ `docs/workable_items.db` $\rightarrow$ the `.workable_items.db` variants.

Usage

```
multitrack_work_binding.sh resolve [--db P] [--file P]... [--files-from  
F|-]  
                                [--ticket ATM-NNN]... [--staged]  
multitrack_work_binding.sh check --branch B [--track T] [--db P]  
                                [--file P]... [--files-from F|-]  
                                [--ticket ATM-NNN]... [--staged]
```

Flag	Meaning
<code>--db P</code>	registry DB path (else <code>\$WI_DB</code> , else the convention paths)
<code>--branch B</code>	the branch the change lands on / is dispatched to (required by <code>check</code> unless <code>--staged</code>)
<code>--track T</code>	the track (<code>track-<N></code>); <code>?</code> or empty $\Rightarrow$ track assertion skipped (honest)
<code>--file P</code>	a repo-root-relative changed path (repeatable)
<code>--files-from F</code>	one path per line (<code>F=-</code> $\Rightarrow$ stdin)
<code>--ticket T</code>	<code>ATM-NNN</code> / <code>SPK-NNN</code> (repeatable); resolved via <code>items.logic_group</code>
<code>--staged</code>	(<code>check</code>) derive branch/track/file-set from git + cwd — the <code>commit_all.sh</code> entry point

resolve — pure lookup

Prints one tab-separated row per **matched** input:

```
<group_id> <dest-branch> <canonical-track> <input>
```

Unmatched inputs print nothing. Exit `0` (exit `2` fail-closed on an unreadable DB).

```
$ multitrack_work_binding.sh resolve --file device/rockchip/atmosphere/remote_host/Foo.kt
mistiq-vader feature/mistiq-vader track-2 device/rockchip/atmosphere/remote_host/Foo.kt
```

check – verdict

```
# commit wrapper entry point
multitrack_work_binding.sh check --staged      # branch/track/files from git+cwd
# explicit
multitrack_work_binding.sh check --branch main --track track-1 --file <p>
# dispatch entry point (hook)
multitrack_work_binding.sh check --branch feature/mistiq-vader --track track-2 --ticket ATM-900
```

- exit 0 — every matched input agrees with (B[, T]) .
- exit 2 — BLOCK (reason on stderr) or fail-closed (unreadable DB / ambiguous scope / malformed ticket).

Outputs / exit codes

Subcommand	stdout	exit
resolve	tab-separated matched rows	0 (2 fail-closed)
check	none	0 allow / 2 BLOCK / 2 fail-closed

The BLOCK reason on stderr names each offending input, its owning group, and the group's canonical branch/track — actionable per §11.4.6.

Edge cases

- **Unclassified input** (no glob owner / no ticket group) ⇒ main-eligible ⇒ skipped (honest partial coverage, §11.4.3).
- **Longest-glob resolution:** the most-specific matching glob wins; a same-length tie between two DIFFERENT groups ⇒ ambiguous ⇒ fail-closed BLOCK (§11.4.6).
- **Track unknown (? /empty):** the track assertion is skipped; the branch assertion still runs (an unknown track is never a proven mismatch).
- **NULL canonical_track** : branch-enforced-only (no track assertion).

- **Pre-v4 DB (no `group_paths`)**: no file owners $\Rightarrow$ file-path enforcement inert; ticket resolution still works.
- **`REPLACE(destination, 'feature:', 'feature/')`** : the DB stores `feature:<slug>` ; the resolver normalises to the `feature/<slug>` branch ref for comparison.

Internal behaviour

1. Resolve the DB; fail-closed BLOCK if `sqlite3 /DB` is missing.
2. Load `logic_groups` (branch + track) and `group_paths` (globs) in one round-trip each.
3. For each file $\rightarrow$ `owner_of_file` (`[[ path == glob ]]` pattern match, longest wins, ambiguity flagged via a global — never a `$(...)` subshell, so the flag survives); for each ticket $\rightarrow$ `ticket_group` (DISTINCT `items.logic_group`).
4. `check` $\rightarrow$ `verdict_for` compares `dest != BRANCH` (the load-bearing clause, the §11.4.191(§6) mutation target) and, when the group is track-pinned and the track is known, `ctrack != TRACK` .

Submodule-context DB resolution (`$ . . / $ . .`)

When this resolver's own cwd is INSIDE a nested submodule (e.g. the constitution submodule itself, edited/committed per §11.4.26), `git rev-parse --show-toplevel` binds to the SUBMODULE's own innermost root — where `docs/workable_items.db` does not exist — so a caller invoking the resolver WITHOUT an explicit `--db` (the detective gate `CM-WORK-TRACK-BINDING-ENFORCED` , or any direct invocation) would fail-closed BLOCK unconditionally, regardless of file-scope classification. The resolver's own auto-discovery therefore walks UP from `git rev-parse --show-toplevel` through every `git rev-parse --show-superproject-working-tree` hop (depth-bounded to 10), probing each discovered OUTER root — OUTERMOST-first — for `docs/workable_items.db / docs/.workable_items.db` . This WIDENS discovery only: a non-nested checkout walks to zero outer roots, so the candidate list is byte-identical to the pre-fix

behaviour, and `$WI_DB / --db` precedence plus the fail-closed-when-nothing-found guarantee are unchanged (§11.4.6 — no weakening). The guard `guard-work-track-binding.sh` also passes an explicit `--db` (defence in depth); both paths coexist.

Related scripts

- `guard-work-track-binding.sh` — the PreToolUse guard that calls this resolver.
- `multitrack_claim.sh` — the §11.4.176 exactly-once group-claim registry (sibling).
- `test_guard_work_track_binding.sh` — the hermetic suite; its section C exercises this resolver directly (track dimension, glob precision, ambiguity, fail-closed).
- `test_multitrack_work_binding_submodule_db.sh` — the §11.4.135 permanent guard for the submodule-context superproject-walk (parent DB resolved from submodule cwd; fail-closed preserved).

Decoupling (`$WI_DB /` `$WI_DB (B)`)

Lives in the constitution submodule, inherited **by reference** (never copied). Zero project literals — reads only the §11.4.93/§11.4.95 universal DB path, cwd, and git. Consumer-owned DATA (the `group_paths` rows, `canonical_track` values) lives in the consuming project's DB.

Last verified: 2026-07-10 — `bash -n + sh -n` clean (§11.4.67); 12/12 resolver smoke assertions PASS; the guard's 34/34 hermetic suite green with the submodule-context superproject-walk added; the resolver resolves the parent DB from submodule cwd (EXIT=0, was fail-closed); the load-bearing `dest != BRANCH` clause proven via an isolated mutation (BLOCK → ALLOW when neutered).